Summary of "Test-Time Compute as Resource Allocation" (10/26)

Ruthesh Thavamani (truthesh), Hang Yeung (yeungh), Eric Zhao (erzhao)

Research direction of the field

Historical Work: Speculation has long been used in computer systems to improve performance by executing work before dependencies are fully resolved. Techniques such as branch prediction in processors, speculative prefetching in memory systems, and optimistic concurrency control in databases exploit predicted outcomes to increase parallelism and hide latency, while validating results and discarding incorrect work when necessary.
Recent Advances: The computational cost and sequential nature of LLM workloads have motivated applying speculation to machine learning systems, where the challenge is to expose parallelism without excessive overhead or sacrificing correctness. Early work such as *Fast Inference from Transformers via Speculative Decoding* (ICML 2023) demonstrated its effectiveness for token generation. However, token-level optimizations do not address dependencies in higher-level workflows, motivating approaches such as *Interactive Speculative Planning* (ICLR 2025) for agent planning and *SpecReason* (NeurIPS 2025) for intermediate reasoning.
Current Papers: We study two works addressing distinct bottlenecks in these workflows. *Speculative Actions: A Lossless Framework for Faster Agentic Systems* (ICLR 2026) predicts and pre-executes future agent interactions to overlap sequential API calls, while *Breaking the Reward Barrier: Accelerating Tree-of-Thought Reasoning via Speculative Exploration* (OSDI 2026) explores reasoning branches before reward evaluations complete. Together, they extend speculation to both external agent interactions and internal reasoning processes.
Subsequent Work: Recent research further explores how to improve higher-level speculation. *SPORK* (arXiv preprint 2026) uses an agent's own model for training-free speculation, while *Speculate While You Reason* (Findings of EMNLP 2026) learns to predict future tool calls through joint training. *Speculative Macro Commit* (arXiv preprint 2026) extends speculation across multiple agent actions, highlighting opportunities to improve prediction accuracy and reduce execution overhead.

Lecture papers

1. Breaking the Reward Barrier: Accelerating Tree-of-Thought Reasoning via Speculative Exploration

Problem: Tree-of-Thought (ToT) reasoning improves LLM performance by exploring multiple paths, but suffers from a reward dependency barrier. In DFS, each rollout waits for reward feedback before the next begins. In BFS, shorter branches wait for longer ones before advancing. These dependencies limit GPU parallelism and KV-cache reuse. Existing inference optimizations target linear reasoning and overlook these bottlenecks.
Proposed solution: SPEX explores promising branches before reward feedback arrives using three techniques: (1) Intra-query speculative selection predicts future branches using reward estimates; (2) Inter-query budget allocation distributes computation based on predicted usefulness, KV-cache reuse, and hardware capacity; and (3) Adaptive early termination stops searches once sufficient confidence is reached. SPEX uses an asynchronous producer-consumer framework built on SGLang. Its main tradeoff is wasted computation from incorrect speculation.
Evaluation: SPEX was tested across four LLMs, four ToT algorithms, and math benchmarks on NVIDIA A6000 and A100 GPUs. It achieved 1.8–3× speedup for DFS and 1.2–1.9× for BFS while generally maintaining accuracy. Combining SPEX with token-level speculative decoding achieved up to 4.1× speedup.
Shortcomings: SPEX relies on predictable search behavior, but prediction accuracy decreases over longer DFS horizons and deeper BFS levels, wasting computation. Its benefits also diminish at larger batch sizes, where GPU utilization is already high and speculation adds resource contention.

2. Speculative Actions: A Lossless Framework for Faster Agentic Systems

Problem: LLM agents face many blocking calls: an LLM invocation, a tool or MCP request, or a human reply, and each call must finish before the next can start, so runs get slowed. Prior agent work applied the idea of

speculation to the agent's own plans. This paper asks whether agents can speculate on any environment interaction to reduce latency without hurting outcomes.
Proposed solution: A fast Speculator (a smaller model or the same model with lighter reasoning) runs alongside an authoritative Actor (an LLM, a tool, or a human). While the Actor works, the Speculator guesses its top k likely results and launches the next step's calls for each guess. If one guess matches, that next step is already underway; if none match, all guesses are discarded. This assumes guesses are right reasonably often and that wrong speculative calls have no visible side effects or can be reversed. With one-step lookahead, latency drops by at most 50%.
Evaluation: There were 4 evals that they tested on.
Chess (TextArena): GPT-5 on high effort is the Actor and GPT-5 on low effort is the Speculator, which predicts the opponent's move. With three guesses, it was right about 55% of the time, cutting time by about 20%.
E-commerce (τ-bench retail): GPT-5 variants and Gemini-2.5-Flash, alone or as an ensemble, predict the user's next request and the API calls it needs. They predicted 22–38% of calls correctly.
HotpotQA: For a Wikipedia-search agent with a Gemini-2.5-Flash Actor, Speculators guess search results so the next query can start early. Top-3 exact-match accuracy reached about 46%.
OS tuning (lossy): A Speculator tunes a Linux scheduler parameter every second, and the Actor overrides it every 10–15 s. The combined system converged in about 13 s versus 200 s for Actor-only, and at lower cost.
Shortcomings: The biggest limitation is scope. Losslessness requires speculative actions to be reversible or sandboxed, which excludes much of what useful agents do, such as writing data or sending messages. The evaluations are chosen to avoid this problem. Also, none of them are applicable agent workloads like coding.

Lecture discussions

SPEX
1. What happens if the reward model isn't well calibrated, especially when branches have similar scores? Poor calibration leads to more wrong speculations and wasted compute, but the final output should remain correct. SPEX uses the confidence gap between the top branches for early termination, but close scores make this harder, and the paper doesn't evaluate these cases.
2. How does speculation overhead affect throughput, and are the evaluated batch sizes realistic? SPEX adds 15-30% compute overhead, which is fine on idle GPUs but becomes expensive under heavy load. The experiments use batches of 8-16, while production systems often reach 64-128, where speedups drop and other requests may get starved.
3. How does SPEX handle GPU resources and KV cache growth? SPEX runs a separate model copy on another GPU, so it depends on having spare compute. Branches share prefixes, which helps with KV cache reuse, but the paper doesn't really address memory pressure at larger scales.
4. Should SPEX's early termination consider tree depth? Yes, because branch confidence can change deeper in the tree. Ignoring depth could prune promising branches too early, but fixing this would require a predictor that looks further ahead.
5. Where would SPEX be most practical? Probably small models, lightly loaded GPUs, or single-user setups like on-device inference. In production serving, the extra compute might be better spent batching requests from different users.

Speculative Actions
1. How do you know a tool call is safe to speculate on, and how do you prevent security risks? Only reversible actions should be speculated, ideally inside a sandbox. Confidence thresholds determine which actions are worth preparing, while anything with external side effects should wait until the commit phase.
2. How can Speculative Actions predict human responses, and does it work with free-form text? Instead of predicting exact words, it guesses likely tool calls or action categories. This works well with MCP and APIs where the options are limited, but it's much harder with unrestricted text.
3. How realistic is the evaluation, especially given correlated prediction errors? The paper doesn't test correlated mistakes and focuses on idealized latency rather than full end-to-end performance. It also only evaluates four scenarios, with no multi-agent experiments, so the actual savings could be smaller.

4. Would Speculative Actions work in environments with many possible next steps, like JEV? Probably not as well. Speculation works best when there are only a few likely actions, while environments like JEV have too many possible directions.

General Discussion

1. How convincing are the evaluations, and could the benchmarks be misleading? Both papers use fairly favorable settings and don't do much to test generalization. SPEX uses three math benchmarks, including the small AIME dataset, part of MATH-500, and relatively easy GSM8K problems with specialized Llemma models. Speculative Actions uses four idealized scenarios without end-to-end or multi-agent evaluation. There was also discussion about benchmark contamination and dataset manipulation being a broader problem in the field.

2. Could combining SPEX and Speculative Actions in a ToT agent give larger speedups? Maybe, but both need extra compute and could compete for GPU resources. Their latency patterns are also different, and neither paper tests the combination.

3. What's the most promising direction for future work? Smaller, specialized speculator models seem promising since large models are expensive to run just for speculation. Databases are another good fit because they already support rollback and separate preparation from commit.

4. What's actually new about these approaches? Speculative Actions predicts tool calls instead of tokens, which is easier because the set of possible actions is smaller. SPEX applies speculation to Tree of Thoughts instead of Chain of Thought, trying to speed up tree search without too much overhead.

Proposal for follow-up work

A promising direction for future work is to develop a mathematical model or reinforcement learning-based scheduler that dynamically tunes speculation depth in production LLM serving systems. While SPEX and Speculative Actions demonstrate that speculative execution can reduce latency, their evaluations focus on relatively simple settings with limited GPU contention. In production, GPUs serve many concurrent requests, meaning that deeper speculation may reduce latency for one request while wasting compute resources and slowing down others. Future work could develop a mathematical cost-benefit model that estimates the expected speedup and excess compute cost of different speculation depths based on prediction accuracy, GPU utilization, batch size, and system load. Building on this model, a reinforcement learning agent could learn to select the optimal speculation depth for each request, maximizing latency reduction while minimizing wasted computation and interference with other requests. The approach could be evaluated under realistic multi-GPU workloads, measuring throughput, tail latency, and compute efficiency to determine whether adaptive speculation provides consistent benefits in production environments.

A reflection on AI usage

Ruthesh: For section 2, I gave Opus 5.5 High the paper and some questions to answer with guidelines on length and content. I then trimmed the response down myself to make it more concise and asked Claude to recheck accuracy/scope.

Eric: For the first paper summary, I initially asked it to just do a summary based on the criteria in the requirements.md file. The prompt was very vague, and there were things in the response that I did not like, such as the format, the length, and some of the language used. So for my next prompt, I gave it more specific instructions such as “make it more concise” and “keep sections to one paragraph.” I found that giving the LLM more specific directions made the output more effective.

Hang: For the research direction and follow-up, I provided GPT 6 medium with the papers and a rough outline for it to challenge and flesh out. Each time, GPT provided verbose output that I had to iterate to clarify and “unslop” the ideas (e.g., checking citations, sharpening ideas, etc.). For example, I used it to find related papers, and I manually verified the references. It was a back-and-forth iteration until it was done.